

Learning that can show its work.

A local AI workspace connecting source evidence, misconception-aware practice and transfer checks.

The screenshot displays the TraceLearn interface, which is a local AI workspace. The top navigation bar shows 'Workspace > Database foundations' and 'Local AI ready' status. The left sidebar contains a 'YOUR WORKSPACE' section with 'Database foundations' selected, and a 'LEARNING PATH' section with 10 items, including '01 SQL NULL' which is currently active. The main content area is titled 'ORIGINAL COURSE - 10 CONCEPT PAIRS' and features a concept check exercise: 'Salaries are 100, 200, and NULL. Which rows survive WHERE salary <> 100?'. The exercise includes four multiple-choice options (A: 200 only, B: 200 and NULL, C: All three rows, D: No rows) and a 'Check my understanding' button. To the right, the 'ORIGINAL TEACHING NOTES' section explains the logic of WHERE clauses and includes a table titled 'SQL - THREE-VALUED LOGIC'.

salary	predicate	WHERE
100	FALSE	excluded
200	TRUE	✓ kept
NULL	UNKNOWN	excluded

ML Empowerment Build Challenge 3.0 - Submission version 1.0

This report describes an implemented prototype and an internal technical benchmark. It does not claim improved grades, validated mastery, real-user adoption, or an independent classroom study.

1. Problem and product

Fluent explanations can make an idea feel familiar without demonstrating that a student can use it. TraceLearn connects an answer to its evidence, surfaces a possible misconception from a selected distractor, and asks a new question that applies the same concept. The initial scope is deliberately narrow: university database foundations.

The first worked scenario is SQL NULL. A learner predicts that salary $\neq 100$ keeps both 200 and NULL. The app checks the answer deterministically, explains that UNKNOWN is filtered by WHERE, opens the original passage, and then tests the same idea using a different condition.

The screenshot displays the TraceLearn application interface. On the left is a sidebar with a 'Database foundations' menu and a 'Learning path' list containing 10 items, with '01 SQL NULL' selected. The main area is titled 'From "I think" to "I understand."' and contains a 'CONCEPT CHECK' section. The question is: 'Salaries are 100, 200, and NULL. Which rows survive WHERE salary $\neq 100$?'. Below the question are four multiple-choice options: A (200 only), B (200 and NULL), C (All three rows), and D (No rows). Option B is selected. A feedback message states: 'A USEFUL GAP TO EXPLORE. Follow the evidence, then try again. The comparison with NULL is UNKNOWN. WHERE keeps only TRUE, so only 200 survives. You may be treating unknown as an ordinary value different from 100.' Below this are buttons for 'null-comparison' and 'null-where'. On the right, a 'BACK TO THE SOURCE' panel shows 'ORIGINAL TEACHING NOTES' for 'WHERE keeps only TRUE', explaining SQL's three-valued logic and providing a table. At the bottom right, there is a 'Browse passages' dropdown showing 'p.1 - WHERE keeps only TRUE' and a 'Source fingerprint' link.

TraceLearn

YOUR WORKSPACE

Database foundations

+ Import your own notes

LEARNING PATH 10

- 01 SQL NULL
- 02 Aggregates
- 03 NOT IN
- 04 Candidate keys
- 05 Functional dependencies
- 06 Attribute closure
- 07 Second normal form
- 08 3NF vs BCNF
- 09 Decomposition
- 10 LEFT JOIN

Learning notebook 2

From "I think" to "I understand."

Find the evidence. Spot the misconception. Test the idea again.

1 Diagnose 2 Trace evidence 3 Transfer & reflect

Practice Ask the source Notebook

01 CONCEPT CHECK

Salaries are 100, 200, and NULL. Which rows survive WHERE salary $\neq 100$?

Choose one answer. A confident mistake is an opportunity to learn.

- A 200 only
- B 200 and NULL
- C All three rows
- D No rows

A USEFUL GAP TO EXPLORE

Follow the evidence, then try again.

The comparison with NULL is UNKNOWN. WHERE keeps only TRUE, so only 200 survives.

You may be treating unknown as an ordinary value different from 100.

null-comparison null-where

Explore this reasoning with local AI

Try the transfer check → Retry

BACK TO THE SOURCE LOCAL

ORIGINAL TEACHING NOTES p. 1

WHERE keeps only TRUE

SQL uses TRUE, FALSE, and UNKNOWN in its three-valued logic. A WHERE clause keeps a row only when its predicate is TRUE; FALSE and UNKNOWN are both filtered out. For salaries 100, 200, and NULL, WHERE salary $\neq 100$ returns only the row with 200. To include missing salaries, write salary $\neq 100$ OR salary IS NULL.

SQL - THREE-VALUED LOGIC

salary $\neq 100$

salary	predicate	WHERE
100	FALSE	excluded
200	TRUE	✓ kept
NULL	UNKNOWN	excluded

null-where

Source text is inspectable. AI interpretations still need judgment.

Browse passages

p.1 - WHERE keeps only TRUE

Source fingerprint

The screenshot shows a scripted software acceptance session, not a study participant.

Intended social value

Students can use their own material without uploading it to a cloud model or paying per API call. Downloads and adequate local hardware are still required. English/Chinese interface and source translations broaden accessibility; built-in questions retain English technical wording. Benefits for learning outcomes remain a hypothesis for a future study.

2. Implementation and boundaries

Layer	Implementation
Reading workspace	React + TypeScript; responsive source and practice panes; cancellation and recovery states.
Local service	Node.js + Express on 127.0.0.1; expected Host/Origin checks; no public deployment.
Documents and provenance	PDF.js or UTF-8 parsing; 10 MB / 50 PDF-page limits; source IDs, page/segment numbers, SHA-256.
Storage	SQLite stores source text, local vectors, questions and practice history. No local database is in the distributable.
Retrieval	0.75 cosine similarity + 0.25 query-token overlap; five highest-ranked passages.
Generation	Qwen3-4B via Ollama; Qwen3-Embedding-0.6B; temperature 0, seed 42, 8192 context, 1200 output-token cap, thinking disabled.
Checks	JSON schema; exact source-quote matching; answer echo/planning rejection; at most one retry.
Practice	20 original passages, 10 diagnostic/transfer pairs; deterministic grading. Imported-source questions are AI-assisted verbatim cloze exercises with a deterministic source-text answer check.

Imported practice was narrowed after a free-form generated item had four incorrect options. In the shipped path, the server selects a phrase from a real source sentence as the keyed answer; the local model supplies three distinct distractors. Completing the gap with the key reconstructs the source sentence exactly. This is source recall, not conceptual transfer. The app distinguishes citation location from entailment. A real quote can still be used to support a wrong claim; exact matching prevents invented quotations, not all hallucination. Prompt-injection resistance is bounded by the model and representative tests.

Key design tradeoff

A narrow, inspectable hybrid retriever reduces the context sent to a small local model. This can improve efficiency but can also omit necessary information. The full-source baseline receives all 20 original passages. Both conditions use the same final prompt and output checks, so this experiment compares context selection, not a different model or an unguarded baseline.

3. Local operation and reproducibility

All inference and data paths remain on the same computer. The browser talks to one local service; the service parses and stores sources, retrieves passages and asks the local model for structured output.

Path	Responsibility
Browser -> Node.js	Learning interface, local import, deterministic grading and report download.
Node.js -> SQLite	Source text, source hash, vectors, questions, attempt and chat records.
Node.js -> Qwen embedding	Create passage vectors once, then embed the current question.
Top five passages -> Qwen3	Produce a structured explanation with exact source excerpts.
Validation -> browser	Accept checked output or return an explicit failure after one retry.

Local runtime measurements

With both app and Ollama running under a macOS policy allowing only localhost outbound connections, real source-grounded answers succeeded. External DNS and direct-IP attempts were denied. A request after restarting Ollama took 6.62 seconds; the warm repeat took 4.17 seconds. Cached SQLite vectors were retained. Ollama reported approximately 6.23 GB allocated across the two loaded models; this is a model allocation snapshot, not peak whole-system memory.

Reproduce locally

```
npm run setup
npm start
http://127.0.0.1:4317
```

Model weights are not bundled. Exact model hashes, package lock, inference settings and all final outputs accompany the source. Tested on Apple M4 Pro with 24 GB memory; other operating systems were not tested.

4. Evaluation design

The author-created dataset contains 80 fixed cases: 40 source-answerable questions, 20 source-unanswerable questions, and 20 multiple-choice questions with predetermined answers. Forty cases form the development split and 40 the holdout split. Related concepts occur in both splits; this is not a test of generalization to unseen courses.

The first pilot was stopped after a separate UI acceptance query returned a question echo with valid quotes. Two pilots exposed question echoes and option-letter leakage. The final v3 prompt requires completed explanations and only uses option-letter instructions for explicit multiple-choice questions. The dataset was not changed. Some pilot holdout requests had run, but their answer content was not used for tuning. Both pilot records and the final run are retained. This is an internal development benchmark, not an independently blinded evaluation.

Final holdout results

Metric	Full source baseline	Hybrid retrieval
Accepted answer on answerable cases	19/20	20/20
Structured insufficient flag	6/10	4/10
Correct extracted MCQ letter	7/10	9/10
Median wall time per request	6.25 s	3.21 s
Requests passing output checks	37/40	38/40

Accepted answer rate is not semantic correctness. MCQ correctness is based on extracted option letters; a formatting miss may count as incorrect even if prose implies the right answer. Expected-source overlap and keyword checks are diagnostic only and are not promoted as accuracy. Quote acceptance is conditioned on passing the validator. Wall time includes retrieval and retries, and can include local workload contention.

Agent-assisted reading of the complete holdout unanswerable outputs found prose refusals on 8/10 for each method, plus 2/10 validation failures. The structured flag undercounts textual refusals; both measures are retained.

Full run

The final run contains 160 requests across both methods. Raw JSONL includes accepted final responses, citations, retrieval passages, available retry records and timings; rejected calls retain error records. Dataset SHA-256:

e2eab651cdbb82544f09a1493b77a059f184442151beab5fe606460e082217ef

See [evaluation/summary.json](#) and [evaluation/raw-results.jsonl](#) for split-level results and individual failures.

5. Interpretation, review and limitations

The product does not treat a correct practice answer as proof of durable understanding. Transfer checks are a concrete opportunity to apply a concept; they are not a validated psychometric instrument.

Agent-assisted review of all 20 source-answerable holdout cases per method judged 17/20 baseline and 18/20 grounded responses fully correct. These are non-independent judgments on an author-created, concept-overlapping dataset; they are not externally validated accuracy or learning gains.

Grounded in-4 used the correct SQL condition but wrote 10:00 instead of 100 in its explanation.

Grounded in-26 gave the formal 3NF definition but added an unsupported comparison to the informal slogan.

Baseline in-28 contradicted itself about BCNF implying 3NF; baseline in-36 contradicted its own source about dependency preservation.

Baseline in-40 failed validation and was not removed from the denominator. Exact quote matching did not prevent all semantic errors.

Structured insufficient flags undercounted prose refusals. On holdout unanswerable cases, both methods declined in prose on 8/10 and returned validation failures on 2/10. In development, grounded out-13 instead invented a university award policy; this is a real unsupported-answer failure.

What failed, and why it matters

The pilot exposed a substantive failure: quote validation alone accepted a response that repeated the question. Output validation was strengthened, but it cannot verify all meaning. The final result set preserves remaining rejected responses, unexpected refusals, answer mistakes, and formatting misses rather than dropping them from denominators.

This benchmark is small, written by the same development process as the course, and lacks independent human scoring. Source support review is agent-assisted and labeled accordingly. The baseline receives a small complete course; results may change for larger documents, other models, other languages or a different retrieval method.

Next research question

A prospective study could compare source-linked diagnostic practice with explanations alone using randomized conditions, independent question review, delayed transfer questions and consented participants. That study was not performed for this submission.

6. Verification and contribution

The final package includes executable unit tests, an isolated HTTP check, actual browser captures, an exported report, and local runtime evidence. SQL NULL examples execute against SQLite; selected candidate-key claims are checked by attribute closure. PDF import tests cover readable PDFs, damaged PDFs, blank/scanned pages and page limits.

Boundary	Evidence
Core implementation	17 unit tests passed; see evidence/unit-tests.log.
HTTP routes	11 checks: hidden answers, grading, history, export, validation, imports, Origin and Host restrictions.
Browser	Actual diagnostic -> citation -> transfer -> notebook workflow; refresh restoration; desktop and narrow layouts; local AI and import paths.
Network independence	See evidence/offline-check.json for process-level localhost-only network validation and its scope.
Reproducibility	Locked package versions, frozen dataset and recorded model digests; exported source archive excludes personal runtime state.

AI contribution disclosure

Codex substantially assisted with design, code, original teaching and evaluation materials, testing, debugging, visual composition and documentation. Qwen provides local runtime responses. Agent-assisted checks are not represented as an independent human review. The public event encourages AI projects; separate organizer approval of AI-assisted code authorship has not been obtained.

Sources and licenses

Original app code and original course materials: MIT. Qwen models: Apache-2.0. Third-party dependencies retain their own licenses; exact package notices accompany the code. Model weights are downloaded separately.

Technical references: ollama.com/library/qwen3:4b; ollama.com/library/qwen3-embedding:0.6b; docs.ollama.com/api/embed; postgresql.org/docs/current/functions-comparison.html; postgresql.org/docs/current/functions-aggregate.html; postgresql.org/docs/current/queries-table-expressions.html.

Submission requirements and scoring: ml-build-challenge-3.devpost.com and [/rules](#). All screenshots show the actual application. Public source and downloadable materials: github.com/FENGJIA666/tracelearn (release v1.0.0). The application runs locally; no hosted inference service or live video is provided.